Preemptive Real Time Operating System for Low Power Microcontrollers

Teodora Cernat
Applied Electronics Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
cernat.do.teodora@student.utcluj.ro

Mihai Daraban
Applied Electronics Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
mihai.daraban@ael.utcluj.ro

Cosmina Corches
Automation Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
cosmina.corches@aut.utcluj.ro

Gabriel Chindris
Applied Electronics Department
Technical University of Cluj-Napoca
Cluj-Napoca, Romania
gabriel.chindris@ael.utcluj.ro

2023 IEEE 29th International Symposium for Design and Technology in Electronic Packaging (SIITME) | 979-8-3503-4411-0/23/$31.00 ©2023 IEEE | DOI: 10.1109/SIITME59799.2023.10431365

***Abstract*—In operating systems (OSs), there are two types of scheduling methods in which CPU resources are allocated: non-preemptive and preemptive scheduling. Embedded system developers generally create applications using a non-preemptive scheduler OS owing to limited resources and better control over the tasks and behavior of the system. With a non-preemptive scheduler, programming tasks and system functionalities are more complex as it does not permit interruptions other than the scheduler. By contrast, a preemptive scheduler allows for easier application development as each task is seen as an independent action or program that can be interrupted at any given time. Therefore, this paper, a scheduler algorithm implementation based on round-robin and priority algorithms for low -power microcontrollers. Combining these two algorithms increases the system performance because of improved resource allocation to the tasks executed.**

Keywords—embedded system, microcontroller, preemptive, operating system, microcontroller, task.

I. INTRODUCTION

At the center of an embedded system is the microcontroller and the application that resides in the memory. To launch the system on the market as quickly as possible, the application software must be available from the early stages of the project, not only for performing tests to catch the bugs in the software but also to help the hardware team in testing the hardware concept [1].

When designing a non-preemptive real-time operating system (RTOS) for embedded systems, much of the time is spent organizing the activity and timing of the task [2]. For a non-preemptive scheduler [3], the tasks need to be completed before a new scheduler call, Fig. 1.

Besides the one controlling the scheduler operation, no other interruptions are permitted because the deterministic response of the system can be affected. Such a condition adds complexity to organizing and developing task actions.

Even if the minor and major cycles of the system are properly designed, there is a risk of task starvation. As mentioned previously, the cooperative operating system (OS) relies on tasks to voluntarily yield control of the CPU. When a task does not cooperate, the CPU is monopolized, which can cause other tasks to wait indefinitely.

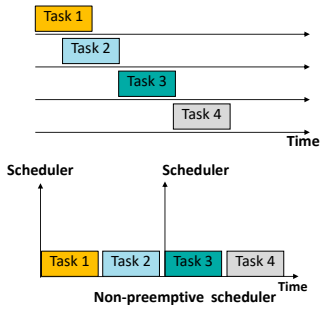

Fig. 1. Non-premptive scheduler.

In such systems, the programmers have a significant responsibility to analyze the OS operation to ensure task cooperation and release of CPU resources.

Preemptive RTOS treats each task as an individual program that can be interrupted either by the peripheral interrupt handlers or the OS scheduler when required to switch to another task, Fig. 2. The challenge in a preemptive scheduler is the context-switching process, which can have varying levels of support for context switching depending on the platform. Additionally, the stack management process must be treated very carefully while saving the context of the current tasks and retrieving the context of the new task.

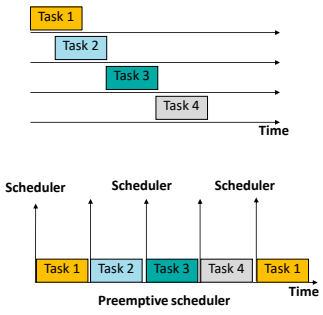

Fig. 2. Preemptive scheduler

However, the preemptive scheduler is preferred in new embedded system designs because it provides better isolation between tasks and improves responsiveness, and fairness in resource allocation [4].

In this paper, the implementation of a preemptive real - time scheduler for low-power microcontrollers is presented. To implement the preemptive scheduler, the round-robin algorithm was used to identify which task needs to be launched when the scheduler is called. Although this approach works well for most of the embedded systems, the proposed approach also incorporates priority management. In scenarios where two tasks are required to be executed simultaneously, priority management allows better control of the tasks that need to run, preventing the scheduler from relying on the order of arrival [5]. Through the priority management, a hybrid operating system was implemented, where a task with high priority can run until it finishes the operation without relinquishing the CPU resources. Such an approach is useful when the system has an action that is safety critical or that might affect the entire system operation.

II. Preemptive Scheduler Algorithms

The type of algorithm used to implement the scheduler of an embedded preemptive OS depends on the system requirements and characteristics (e.g., real-time constraints, task priorities, system resource constraints, and the complexity of the scheduling requirements).

One of the most common preemptive scheduler algorithms is priority-based scheduling, Fig. 3. Such an algorithm would suit systems where the tasks have well-defined priorities. The scheduler will peek to launch the highest-priority-ready task, which will ultimately help meet the real-time deadlines. Priority scheduling is a widely- used algorithm because of its simplicity of implementation. However, as with any scheduling algorithm, priority scheduling has the following disadvantages:

- Starvation: Without careful design, it is possible that high-priority tasks may not relinquish CPU resources if they are constantly ready to run. In this case, low-priority tasks may not receive the CPU time needed, which can affect real-time deadlines.
- Task prioritization challenges: Determining the right priority order is crucial to meet real-time requirements. This process can be challenging in systems with a large number of tasks.
- Complexity in handling task dependencies: If there are tasks with complex dependencies or that require a specific execution order, such a scenario is difficult to implement in priority-based scheduling.

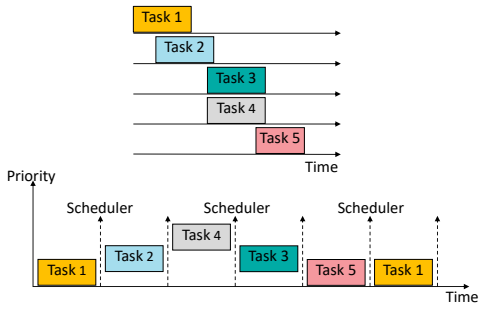

Fig. 3. Preemptive priority-based scheduling

An alternative to priority-based scheduling is the earliest deadline first (EDF) scheduling algorithm [6]. In this approach, the scheduling algorithm will launch the task with the earliest deadline. Although the EDF algorithm can yield good results as a scheduling strategy for real-time systems, it has some disadvantages that, for low-power embedded systems, may be perceived as drawbacks:

- Complexity of implementation: Because it requires monitoring and updating the task priority based on their deadlines, EDF can be more complex to implement than fixed-priority or round-robin algorithms.
- Memory requirements: Compared with other scheduling algorithms, EDF may require additional memory to store information about task deadlines, periods, and scheduling parameters.
- High energy consumption: Because EDF needs to adjust task priorities continuously, it translates into a runtime overhead and frequent context switches. All the previous actions translate into higher energy consumption because of the increased power overhead of switching CPU states.

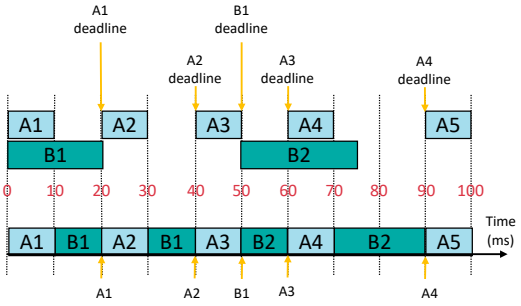

Fig. 4. Earliest deadline first scheduler

When analyzing potential scheduler algorithms, it becomes apparent that there is no algorithm suitable for every system without some drawbacks. A light scheduling algorithm, such as round-robin scheduling, may be simple to implement and ensure fairness among tasks but lacks control over real-time constraints. However, adding complexity, as in the case of priority scheduling, may aid in meeting real-time deadlines but can impact fairness or give rise to issues, such as starvation during task execution.

First, a scheduling algorithm that belongs to a class of schedulers called energy-efficient scheduling (EES) appears promising for low-power embedded systems. Typically, such an algorithm tries to prioritize tasks to meet deadlines while minimizing energy consumption. EES algorithms share similarities with the EDF algorithm. However, the algorithms track the energy usage to make decisions, accordingly, making them perfect for battery-power embedded systems.

However, such algorithms can be moderate to highly complex because there is a need for a task characterization from a power consumption perspective (e.g., different power modes or energy consumption rates) to implement and design the scheduling strategy. Hardware support for power management features is crucial because systems, such as dynamic voltage and frequency scaling or switching between different levels of power modes, can simplify the implementation of EES.

For low power microcontrollers with few resources and a simplified architecture, priority scheduling combined with round robin remains one of the practical approaches through which fairness and responsiveness of an embedded system is achieved. Usually when combining round robin scheduling with priority-based scheduling, the round robin algorithm is used in deciding which task to run next when there are tasks of equal priority. The proposed approach is to use round robin to organize the tasks and when a task with higher priority needs to be run, the scheduler will switch to the higher priority task. To prevent starvation because of the higher priority task, a counter is used to switch between the type of algorithm used by the scheduler. Details of the counter implementation are provided in the next section.

III. Proposed Hybrid Algorithm

To implement the preemptive OS, the task control block (TCB) needed to define and control the tasks' proprieties was defined. The fields defined in TCB are as follows:

- task stack pointer – a pointer needed to point to the task's designated stack-up and implicitly the starting point;
- task status – field used in checking the task status (e.g., stopped, ready, waiting, and running);
- task period – the time interval when the task needs to run;
- task last run – the time since the last run; once this time is greater than or equal to the task period time, the task needs to be launched.

To implement the priority scheduler, two more fields were added to TCB:

- task priority – field used in specifying the priority of some of the tasks;
- task consecutive runs – field used to track how long the task uses the system computing resources.

The scheduling algorithms are launched through a timer interrupt handler, which is set to be triggered at time intervals equal to the OS minor cycle (Fig. 5). Based on the scheduling algorithms output results, each embedded system task will receive a time slice to execute the code. Because in preemptive OS, the context switching needs to be controlled by OS, each task needs to be designed as an individual program with an infinite loop to avoid uncontrolled context switch, as the task -associated stack pointer is not able to recall the context switch. To improve the system power consumption, once the task finishes its execution, the system is put to sleep until the end of the time slice.

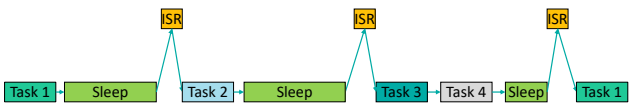

Fig. 5. Preemptive task execution

TCB field task status, task period, and task last run help control the round-robin algorithm for identifying and launching a new task (Fig. 6). The algorithm checks each task individually to see if their launching time has come based on the task period timing. The tasks with priority set to zero are managed by the round-robin scheduling algorithm. The tasks with higher priorities can also be launched through the round-robin algorithm if the running condition is fulfilled when checked.

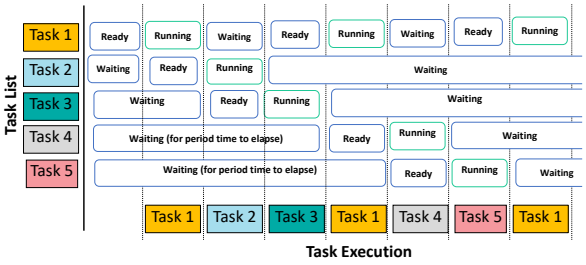

Fig. 6. Round Robin algorithm execution based on the task status and task period property

After the round-robin algorithm decision, a second check is done using the priority scheduling algorithm over the tasks with priority settings. At this time, it is checked if there is not another task with a higher priority that wants to run. The priority comparison starts with the round-robin algorithm as a reference, which can be a task with or without a priority. Once a higher-priority task is found, then that becomes a reference for the remaining tasks to compare Fig 7.

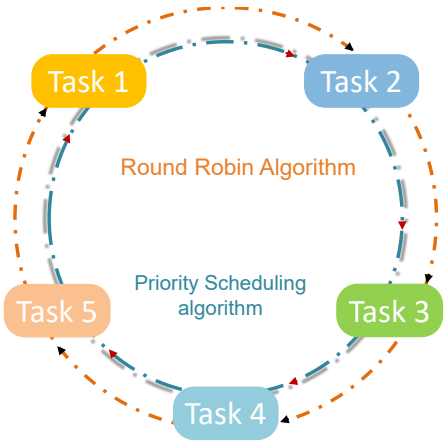

Fig. 7. High priority scheduler checking for a task with a higher priority

Next, the consecutive run field is used to decide between tasks with the same priority. If the tasks have the field propriety set to zero (i.e., neither one of the tasks was run recently), the algorithm will select the first task found. Otherwise, the task with the smallest value stored in the task consecutive run field will be launched. This approach ensures that tasks with the same priority are treated in the same manner.

At first glance, it might seem that the tasks with the priority field are preferred, and system starvation can occur. To prevent such a scenario, a threshold limit was designed for the priority scheduler algorithm. If the sum of the consecutive run field of the tasks launched using the priority-based algorithm is over a threshold, then the high-priority tasks are no longer selected, and the system reverts to the round-robin algorithm decision. In such a scenario, the task execution is

resumed from the selected task using the round-robin algorithm before the priority algorithm. An additional check is done that the task does not have a priority and is executed through the priority scheduling algorithm. If the previous condition is not fulfilled, the next task that meets the requirements is selected.

Through this mechanism, the scheduler ensures the system's farness is not affected by the high-priority tasks. Once the round-robin scheduling fully checks the tasks to ensure that each of them has the chance to run, the priority scheduling is re-enabled.

IV. Conclusions

This paper presented a hybrid scheduling algorithm for embedded systems. The proposed algorithm can be used in real-time embedded systems with soft timing requirements. The algorithm can be used in hard RTOS if few tasks are controlled through the priority scheduler and the timing periods and priority levels are carefully designed.

The algorithm would benefit from having knowledge of the shortest deadline, which may increase the overall complexity of the system. Through the counter field, a mix of functionality and resources has been achieved.

References

[1] Z. Ma, J. S. Collofello and D. E. Smith-Daniels, "Improving software on-time delivery: an investigation of project delays", 2000 IEEE Aerospace Conference. Proceedings, vol. 4, pp. 421–434, 2000;

[2] M. Nahas and A. M. Nahhas, "Ways for Implementing Highly-Predictable Embedded Systems Using Time-Triggered Co-Operative (TTC) Architectures," in Embedded Systems, K. Tanaka, IntechOpen, pp. 3-30, 2012;

[3] H. K. Tang, P. Ramanathan and K. Compton, "Run-Time Scheduling of Non-preemptible Real-Time Tasks in Heterogeneous Systems', in 27th International Conference on VLSI Design and 2014 13th International Conference on Embedded Systems. IEEE, pp.168–173, 2014.

[4] P. M. Castro, I. Harjunkoski and I. E. Grossmann, "Expanding RTN discrete-time scheduling formulations to preemptive tasks," Computer Aided Chemical Engineering, Vol. 44, pp. 1225-1230, 2018.

[5] L. Cheng, Y. Tao and S. Piao, "The design of embedded Operating System for vehicle Internet of Things", The 2nd International Seminar on Computer Science and Engineering Technology (SCSET) 2020, October, Shanghai, China;

[6] Xiaojie Li and Xianbo He, "The improved EDF scheduling algorithm for embedded real-time system in the uncertain environment," 2010 3rd International Conference on Advanced Computer Theory and Engineering(ICACTE), Chengdu, pp. V4-563-V4-566, 2010.